

Focus & Feel

Project Documentation

Skills: Programming with Advanced Computer Languages

University of St. Gallen (HSG), MacFin Programme

Student: Balint Nussbaumer

Language: JavaScript

June 2026

Github link: <https://github.com/OxBaumer/focus-feel>

1. Project Overview

Focus & Feel is a web app that works like a study timer. You type what you are working on, start the timer, and stop it when you finish. After stopping, you choose how the session felt (Good, Okay or Bad) and can add a short note.

Each session is sent to a server and saved in a database, so the data is kept centrally and stays available, even from another computer. A dashboard shows how many sessions you have done, the total minutes, a simple bar chart, and a list of all sessions with their mood.

2. Features

- Stopwatch timer shown as minutes and seconds (for example 02:05)
- One Start and Stop button that does both jobs
- A task name has to be entered before the timer can start
- Mood choice after stopping: Good, Okay or Bad
- Optional short note for each session
- Sessions are saved in a database on the server
- Dashboard with the number of sessions and the total minutes
- Two simple charts: time worked per day, and average mood per day
- A list of all sessions showing the mood as an emoji and the note

3. How It Is Built

The project has three parts: the web page that the user sees, a small server, and a database. They work together over the internet.

Part	What it is used for
HTML and CSS	The structure and the look of the page
JavaScript	The timer, the buttons and the dashboard, inside index.html
Bun server	A small server (server.js) that sends the page and answers requests
SQLite database	A simple database (set up in db.js) that stores the sessions
Railway	The online service that runs the server and keeps the database

This project is not a single .js file because it is a web app. A web app always needs an HTML page (the part the browser shows on screen) together with JavaScript (the part that reacts when you click). A lone JavaScript file cannot show a page with an input box, a timer and buttons on its own.

So the page is written in HTML and CSS, and the actions (the timer, choosing a mood, saving) are written in JavaScript inside that page. Because the app also stores the sessions in a real database, there is in addition a small server (server.js) and the database setup (db.js). That is why the project has a few files and not only one.

4. Project Structure

File	Role
index.html	The web page and the browser JavaScript
server.js	The server: sends the page, saves and reads sessions
db.js	Opens the SQLite database and creates the table
package.json	Tells how to start the server
Dockerfile	Tells Railway to run the project with Bun
Focus_Feel_Documentation.docx	This documentation

5. The Main Parts

The browser side (inside index.html) is split into short functions:

Function	What it does
showTime()	Shows the seconds on screen as minutes and seconds
tick()	Adds one second; it is called every second by setInterval
startStop()	Starts the timer, or stops it and shows the mood question
pickMood()	Remembers which mood button was clicked
saveSession()	Sends the session to the server, then resets the page
loadSessions()	Asks the server for all sessions and shows them
showStats()	Calculates the totals and builds the chart and the list

The server (server.js) answers two requests:

Request	What the server does
GET /api/sessions	Reads all sessions from the database and sends them back
POST /api/sessions	Saves a new session into the database

Each session is stored as one row in the database table:

```
CREATE TABLE sessions (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  task TEXT,  
  seconds INTEGER,
```

```
mood TEXT,  
note TEXT  
);
```

6. How It Works, Step by Step

1. You open the app. The page calls `loadSessions()`, which does a GET request to the server. The server reads the sessions from the database and sends them back, and the dashboard shows them.
2. You type a task name and click Start. The function `setInterval` calls `tick()` once every second, so the timer counts up.
3. You click Stop. `clearInterval` stops the counting. If some time was counted, the mood question appears.
4. You pick a mood, optionally write a note, and click Save session. The page sends the session to the server with a POST request.
5. The server runs an SQL INSERT to save the session in the database. The page then loads the updated list again, so the dashboard shows the new session.

7. How to Run It

Important: this version uses a server and a database, so you cannot run it by just double clicking `index.html`. The page needs the server to load and to save the sessions. There are two ways to run it:

Online (easiest): open the public Railway link for the project in a web browser. The server is already running there.

On your own computer:

1. Install Bun (the JavaScript tool, from bun.sh).

```
bun server.js
```

2. Open the address it shows (`localhost:3000`) in a web browser.

Note: if you open `index.html` on its own from the file system, the timer and the buttons show up, but saving will not work, because there is no server for the page to talk to.

8. Deployment on Railway

The code is on GitHub. Railway is connected to it and rebuilds the project automatically every time new code is pushed.

- Railway uses the Dockerfile, which runs the project with Bun
- A storage Volume is mounted at `/data` so the database file is kept between deployments
- A variable `DB_PATH` set to `/data/focus.db` tells the server to store the database on that volume

This is the line in `db.js` that reads the path:

```
var db = new Database(process.env.DB_PATH || "focus.db");
```